# Indicode Front-End 40-Item Code Audit - v0.9.8 (pre-release)

== Compiler Front-End: Remaining Work and Completion Plan

Front-end package version: 0.9.8 (pre-release; not a complete compiler).

Status date: 2026-11-03 Scope: Indicode compiler front end only. This code audit replaces the previous all-area blueprint PDF.

For the consolidated progress, completed-work summary, percentage breakdown, and remaining roadmap, see the front-end status report and its PDF.

-- What “100%” means here

The front end reads source, checks it, reports source-aware errors, and produces a semantically valid, independently verified Indicode HIR. It does not generate LLVM IR, machine code, executables, or run programs.

The current weighted front-end readiness estimate is 97.25%, displayed as 97%. The estimate includes broader conformance and release-readiness gaps; it is not the percentage of the catalogue or a measure of how much of a complete compiler exists.

The old PDF described a 777-concept catalogue across the whole language/compiler project. Only 40 catalogue entries (IDs 445–484) are in its compiler-front-end section. In the current implementation audit:

| Front-end catalogue status | Count |
| --- | --- |
| Implemented and focused-tested at the stated v1 or public-API acceptance level | 40 |
| Partially implemented at that acceptance level | 0 |
| No implementation at that acceptance level | 0 |
| Front-end catalogue total | 40 |
| Still unfinished (partial + not implemented) | 0 |

These inventory counts and the 97.25% weighted estimate measure different things. Some catalogue entries are library APIs and are not integrated into the single-file source grammar, semantic analyzer, CLI, or an LSP. The remaining 737 entries in the full catalogue are outside this front-end-only plan; this is not a claim that they are complete.

-- Code audit of all 40 front-end entries

“Implemented and tested” means source code and focused test evidence exist at the item’s stated v1 or public-API scope. It does not claim that every API has source-language syntax or production-grade integration. The evidence below refers to implementation files and CTest targets rather than treating an earlier report as proof.

| ID | PDF concept | Audit status | Code/test evidence and scope boundary |
| --- | --- | --- | --- |
| 445 | Source manager | Implemented/tested | source.cpp, source.hpp; source_tests. Owns source files and assigns IDs. |
| 446 | Source buffer | Implemented/tested | SourceManager owns multiple source buffers; tested for stable access and bounds. |
| 447 | Source span | Implemented/tested | SourceLocation/SourceSpan in source.hpp; source boundary tests. |
| 448 | Line table | Implemented/tested | Indexed CR/LF/CRLF mapping in source.cpp; source_tests. |
| 449 | Column mapping | Implemented/tested | Byte offsets, UTF-8 scalar columns, and four-column tabs; source_tests, lexer_tests, diagnostic_tests. Terminal display width remains unsupported. |
| 450 | UTF-8 validator | Implemented/tested | lexer.cpp; invalid source encodings in tokens, strings, and comments tested in lexer_tests. |
| 451 | Token kind | Implemented/tested | TokenKind and 17 keyword variants in token.hpp; token inventory tested in lexer_tests. |
| 452 | Token payload | Implemented/tested | Typed keyword, integer, float, and string payloads in token.hpp/lexer.cpp; payload/spelling/span cases in lexer_tests. |
| 453 | Lexer state | Implemented/tested | EOF and malformed-token recovery in lexer.cpp; focused and parser-robustness tests. |
| 454 | Trivia preservation | Implemented/tested | Whitespace and line/nested-block comments retain spelling and spans in lexer.cpp; lexer_tests. |

455 | Parser state | Implemented/tested | Explicit parser cursor and progress guard in parser.cpp; parser_tests, parser_robustness_tests.
456 | Recursive descent parser | Implemented/tested | parser.cpp parses declarations, statements, generic functions, ownership modes, and effects; grammar_tests, parser_tests. Modules/import syntax and overload declarations are not parsed.
457 | Pratt expression parser | Implemented/tested | parser.cpp precedence table; every binary operator, prefix nesting, calls/index/member tested in grammar_tests.
458 | Grammar version | Implemented/tested | grammar.hpp exposes 1.0.0; grammar/spec agreement tested in grammar_tests. Package version 0.9.8 is separate.
459 | AST context | Implemented/tested | ast.hpp ownership plus AST_CONTEXT_V1.md; parser tests verify AST survives parser/source destruction.
460 | AST node arena | Implemented/tested | Deliberate unique_ptr ownership in ast.hpp; ast_scalability_tests parses/destroys 10,000 statements. No arena is used.
461 | AST visitor | Implemented/tested | ast_visitor.hpp/.cpp; ast_visitor_tests covers each declaration/statement/expression kind.
462 | AST printer | Implemented/tested | ast_printer.cpp; deterministic golden test in ast_printer_tests.
463 | AST serializer | Implemented/tested | ast_json.cpp emits schema version 2; structure/schema checks in parser_tests, grammar_tests, and cli_ast_json.
464 | Syntax recovery | Implemented/tested | Declaration, statement, and delimited-expression recovery in parser.cpp; parser_tests, parser_robustness_tests.
465 | Error synchronization | Implemented/tested | Synchronization routines in parser.cpp, documented in ERROR_RECOVERY_V1.md; malformed-input/recovery tests.
466 | Symbol identity | Implemented/tested | Stable IDs assigned in sema.cpp and carried in hir.cpp; sema_tests, hir_tests.
467 | Scope table | Implemented/tested | Lexical scopes and namespaces in sema.cpp; sema_tests and independent HIR scope checks in hir_tests.
468 | Module resolver | Implemented/tested API | resolution.cpp maps dotted names to .indi paths and diagnoses invalid/missing/ambiguous matches; resolution_tests. Not wired to source syntax/CLI.
469 | Import resolver | Implemented/tested API | ImportResolver handles aliases, public exports, missing modules, and cycles in resolution.cpp; resolution_tests. Not wired to source syntax/CLI.
470 | Overload candidate set | Implemented/tested API | Exact signature and named-argument candidate selection in resolution.cpp; resolution_tests. Not wired to parser/analyzer calls.
471 | Name binding | Implemented/tested | sema.cpp resolves supported values, functions, records, fields, and types; positive/negative identity checks in sema_tests, hir_tests.
472 | Type checker | Implemented/tested | sema.cpp plus hir.cpp; sema_tests, hir_tests cover expressions, calls, conversions, records, collections, tensors, and invalid inputs.
473 | Constraint solver | Implemented/tested | sema.cpp infers generic parameters and checks Numeric/Integer; positive/negative cases in sema_tests.
474 | Type substitution engine | Implemented/tested | sema.cpp substitutes inferred generic types in supported patterns/returns; sema_tests, hir_tests. No explicit call-site type arguments.
475 | Constant evaluator | Implemented/tested | const_eval.cpp; sema_tests cover short-circuiting, overflow, division by zero, and the 256-node limit.
476 | Control-flow validation | Implemented/tested | sema.cpp checks conditions, counts, reachability and return typing; sema_tests, hir_tests. Loops are conservatively not assumed to execute.
477 | Definite return analysis | Implemented/tested | sema.cpp and independent hir.cpp validation; nested branches, unreachable statements, and loop paths tested.
478 | Effect checker | Implemented/tested | effects.cpp infers io, allocation, mutation and transitive calls; annotation errors tested in sema_tests, summaries checked in hir_tests.
479 | Ownership checker | Implemented/tested | ownership.cpp checks mode matching, moves, use-after-move, and call-scoped borrow conflicts; positive/negative cases in sema_tests, metadata in hir_tests. Not a lifetime/escape checker.
480 | Tensor shape checker | Implemented/tested | Exact element/rank/dimension checks in sema.cpp/hir.cpp; tensor tests in sema_tests, hir_tests, and CLI example. No tensor operations/device rules.
481 | Diagnostic engine | Implemented/tested | diagnostic.cpp; diagnostic_tests cover text, notes, fix-its, escaping, order, and locations.
482 | JSON diagnostics | Implemented/tested | diagnostic.cpp emits schema version 1; golden and CLI checks in diagnostic_tests, cli_json_diagnostics, cli_json_parser_fixit.

483 | Language server index | Implemented/tested API | symbol_index.cpp indexes HIR symbols, references, scopes, and revisions; symbol_index_tests. It is not an LSP server and requires HIR snapshots.
484 | Incremental parse support | Implemented/tested | incremental_parser.cpp reuses unchanged top-level fragments across edits; incremental_parser_tests compare edits/recovery with clean parses.

-- Remaining work and next milestones

The 40 catalogue entries are implemented and focused-tested at their defined scope. The following work is outside that completed item count and is why the weighted front-end readiness estimate is still 97.25%:

1. Language integration: decide whether module/import and overload syntax should enter a future grammar version, then connect the existing resolver APIs to source loading, semantic analysis, HIR identities, and CLI workflows. The current grammar remains single-file; the resolver APIs are directly callable library features.
2. Editor tooling: evolve the HIR symbol/reference index API into an editor or language-server integration if required. The current index is not an LSP and is rebuilt from HIR snapshots.
3. Conformance and robustness: expand requirement-linked tests beyond the focused suite; add coverage-guided lexer/parser fuzzing and sanitizer gates.
4. Release readiness: add reproducible offline CI/release jobs, a compatibility policy for package and JSON formats, and broader supported- compiler/platform checks.
5. Language/backend scope: document additional language/runtime semantics before adding them. LLVM IR, machine-code generation, execution, tensor operations, and device backends are not part of this package.

-- What “40/40” and “97.25%” mean

The 40/40 result is a catalogue item count backed by implementations and focused tests; it includes standalone resolver and HIR-index APIs whose integration boundaries are stated above. The 97.25% figure is a separate, judgment-based weighted readiness estimate that includes broader conformance and release acceptance. Neither result means the full language or complete compiler is finished. This package is a pre-release compiler front end, not a machine-code compiler or runtime.

-- Verification

The current acceptance suite contains 23 CTest cases. A fresh Release configure/build and all 23 tests pass. The suite covers source mapping, lexing/trivia, parser/AST/grammar, semantic analysis and HIR verification, diagnostics, symbol indexing, module/import/overload APIs, incremental parsing, CLI output and installed-package consumption. Passing tests establish the covered behaviors, not exhaustive language conformance.